Mud 게임 리포트

전남대학교
CHONNAM NATIONAL UNIVERSITY



과목명	C++ 프로그래밍 및 실습
담당교수	김미수 교수님
학과	인공지능학부
학년	2학년
학번	223274
이름	김영광
제출일	2026.11.02

1. 서론

1.1 프로젝트 목적 및 배경

제공된 기존 Mud 게임 코드를 기반으로 7주차까지 배운 내용에 대한 실습을 위한 진행 및 요구사항 기능 만들기

1.2 목표

7주차까지 배운 것을 토대로 간단한 Mud 게임 구현 및 기존 코드 함수화 하는 연습과 기존 코드 이해하는 것을 목표

2. 요구사항

2.1 사용자 요구사항

a. 유저는 체력 20을 가지고 게임을 시작
b. 사용자가 이동할때마다 체력 1씩 감소
c. 처음 명령문을 입력 받을 때 마다 HP 함께 출력
d. 무기/갑옷, 포션, 적을 만났을 때 그에 대한 메시지를 출력

2.2 기능 계획

a. 사용자 체력을 20으로 초기화
b. 사용자가 유효한 맵안에서 이동 했을 시에 채력 1씩 감소
c. 상하좌우 입력 받기전에 현재 체력 메시지 출력
d. 현재 사용자 위치를 받고 맵에 무기/갑옷, 포션, 적 있는 지 확인 후 맞는 메시지 출력

2.3 함수 계획

a. 메인 함수: 메인 함수 안에서 사용자 체력 20으로 초기화
b. moveXx(상하좌우): 유효한 맵안에서 움직이면 체력 1씩 감소
c. isValidUserHp: 사용자 입력 전 현재 체력 출력 후 체력이 0이하면 false 반환
d. checkState: 도착지에 아이템, 적, 포션 확인 후 유저 체력 변경

3. 설계 및 구현

3.1 설계

a. 메인함수: userHp = 20으로 사용자 체력 20으로 초기화. 메인 함수 시작 시 사용자 체력 20으로 초기화 한다

b. moveXx: 메인함수에서 상,하,좌,우에 대한 사용자 입력을 분기 하고 각 분기에 맞는 moveXx함수 실행

c. isValidUserHp: 현재 사용자 체력을 파라미터로 받고 출력 후 사용자 체력이 userHp< 1이하면 false 반환으로 메인 함수에서 반환 값이 false이면 프로그램 종료

d. checkState: 맵 정보와 사용자 위치와 유저 체력을 파라미터로 받은 후 각 사용자 자리에 아이템,적,포션, swicth문으로 분기 처리 후 각 위치에 맞는 체력 조정처리

3.2 기능 별 구현 사항

a. 메인함수 안에서 사용자 체력 20으로 초기화

```
// 메인  함수
int main() {
    // 0은 빈 공간, 1은 아이템, 2는 적, 3은 포션, 4는 목적지
    int map[mapY][mapX] = { {0, 1, 2, 0, 4},
                            {1, 0, 0, 2, 0},
                            {0, 0, 0, 0, 0},
                            {0, 2, 3, 0, 0},
                            {3, 0, 0, 0, 2} };


    // 유저의 위치를 저장할 변수
    int user_x = 0; // 가로 번호
    int user_y = 0; // 세로 번호

    //유저 HP
    int userHp = 20;
```

b. 사용자가 이동할때마다 체력 1씩 감소

moveXx(상하좌우): 유효한 맵안에서 움직이면 체력 1씩 감소

moveUp: 유효한 맵안에서 위로 움직이는 함수, 위로 증가는 배열 기준으로 첫번째 배열 인덱스가 감소하는 것이므로 user_y -= 1감소 후 유효성 체크 후 체력 증감 적용

유효한 움직임이었으면 유저 체력 1 감소

```
//위로 움직이는 함수
void moveUp(int map[][mapX], int& user_x, int& user_y, int& userHp) {
    // 위로 한 칸 올라가기
    user_y -= 1;


    if (!checkXY(user_x, mapX, user_y, mapY)) {
        cout << "맵을 벗어났습니다. 다시 돌아갑니다." << endl;
    }
    else {
        cout << "위로 한 칸 올라갑니다." << endl;

        displayMap(map, user_x, user_y);

        // user hp 1 감소
        userHp -= 1;
    }
}
```

moveDown: 유효한 맵안에서 위로 움직이는 함수, 아래로 움직이는 것은 배열 기준으로 첫번째 인덱스가 증가하는 것이므로 user_y += 1 증가 후 유효성 체크 후 체력 증감 적용, 유효한 움직임이었으면 유저 체력 1 감소

```
//아래로 움직이는 함수
void moveDown(int map[][mapX], int& user_x, int& user_y, int& userHp) {
    Implement with Codex
    // TODO: 아래로 한 칸 내려가기
    user_y += 1;

    if (!checkXY(user_x, mapX, user_y, mapY)) {
        cout << "맵을 벗어났습니다. 다시 돌아갑니다." << endl;
        user_y -= 1;
    }
    else {
        cout << "위로 한 칸 내려갑니다." << endl;
        displayMap(map, user_x, user_y);

        // user hp 1 감소
        userHp -= 1;
    }
}
```

moveLeft: 유효한 맵안에서 위로 움직이는 함수, 왼쪽으로 움직이는 것은 배열 기준으로 두번째 인덱스가 감소하는 것이므로 user_x -= 1 감소 후 유효성 체크 후 체력 증감 적용, 유효한 움직임이었으면 유저 체력 1 감소

```
//왼쪽로 움직이는 함수
void moveLeft(int map[][mapX], int& user_x, int& user_y, int& userHp) {
    Implement with Codex
    // TODO: 왼쪽으로 이동하기
    user_x -= 1;


    if (!checkXY(user_x, mapX, user_y, mapY)) {
        cout << "맵을 벗어났습니다. 다시 돌아갑니다." << endl;
        user_x += 1;
    }
    else {
        cout << "왼쪽으로 이동합니다." << endl;
        displayMap(map, user_x, user_y);

        // user hp 1 감소
        userHp -= 1;
    }
}
```

moveRight: 유효한 맵안에서 위로 움직이는 함수, 오른쪽으로 움직이는 것은 배열 기준으로 두번째 인덱스가 증가하는 것이므로 user_x += 1 증가 후 유효성 체크 후 체력 증감 적용, 유효한 움직임이었으면 유저 체력 1 감소

```
//오른쪽로 움직이는 함수
void moveRight(int map[][mapX], int& user_x, int& user_y, int& userHp) {
    Implement with Codex
    // TODO: 오른쪽으로 이동하기
    user_x += 1;

    if (!checkXY(user_x, mapX, user_y, mapY)) {
        cout << "맵을 벗어났습니다. 다시 돌아갑니다." << endl;
        user_x -= 1;
    }
    else {
        cout << "오른쪽으로 이동합니다." << endl;
        displayMap(map, user_x, user_y);

        // user hp 1 감소
        userHp -= 1;
    }
}
```

c. isValidUserHp: 유저 체력 파라미터로 받은 후 현재 HP 출력 및 Hp < 1 미만이면 false 반환

```cpp
// userHp 0이하시 실패 출력 후 종료하는 함수
bool isValidkUserHp(int& userHp) {
    cout << "현재 Hp: " << userHp << " ";

    // Hp 1이하면 false 반환
    if (userHp < 1) {
        cout << "실패";
        return false;
    }

    return true;
}
```

d. checkState: 도착지에 아이템, 적, 포션 확인 후 유저 체력 변경

유저 위치 별로 swich문으로 아이템, 적 , 포션 분기 적과 포션이 있으면 맞는 체력 증감 적용

```cpp
// 도착지에 아이템.적.포션 확인 후 출력 및 유저 체력 변경
void checkState(int map[][mapX], int user_x, int user_y, int& userHp) {
    // 0은 빈 공간, 1은 아이템, 2는 적, 3은 포션
    switch (map[user_y][user_x]){
    case 0:
        //빈공간
        break;
    case 1:
        cout << "아이템이 있습니다. " << endl;
        break;
    case 2:
        cout << "적이 있습니다. " << endl;
        userHp -= 2;
        cout << "HP가 2 줄어듭니다. " << endl;
        break;
    case 3:
        cout << "포션이 있습니다. " << endl;
        userHp += 2;
        cout << "HP가 2 증가합니다. " << endl;
        break;
    }
}
```

4. 테스트

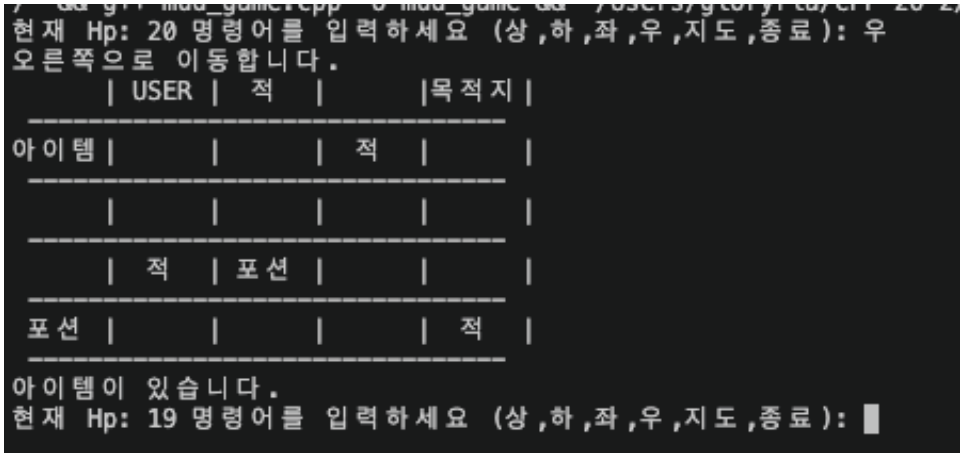
4.1 기능 별 테스트 결과

a. 메인함수 유저체력 20으로 초기화

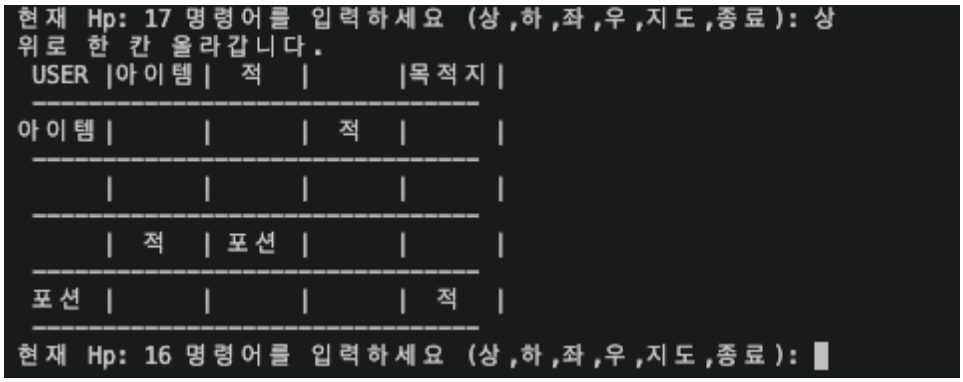


b. moveXx(상하좌우): 유효한 맵안에서 움직이면 체력 1씩 감소 및 유저 위치 이동

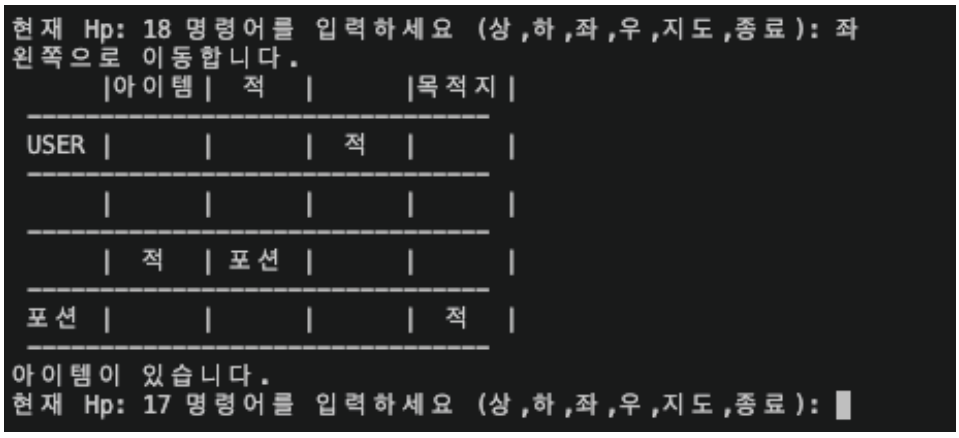
moveRight: 우 입력후 첫 시작 위치에서 오른쪽 한칸 이동 확인



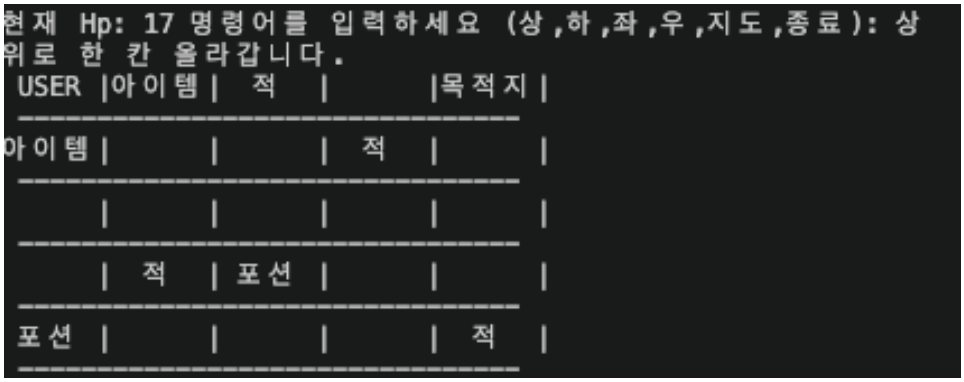
moveLeft: 좌 입력후 첫 시작 위치에서 왼쪽한칸 이동 확인



moveDown: 하 입력후 첫 시작 위치에서 아래로 한칸 이동 확인

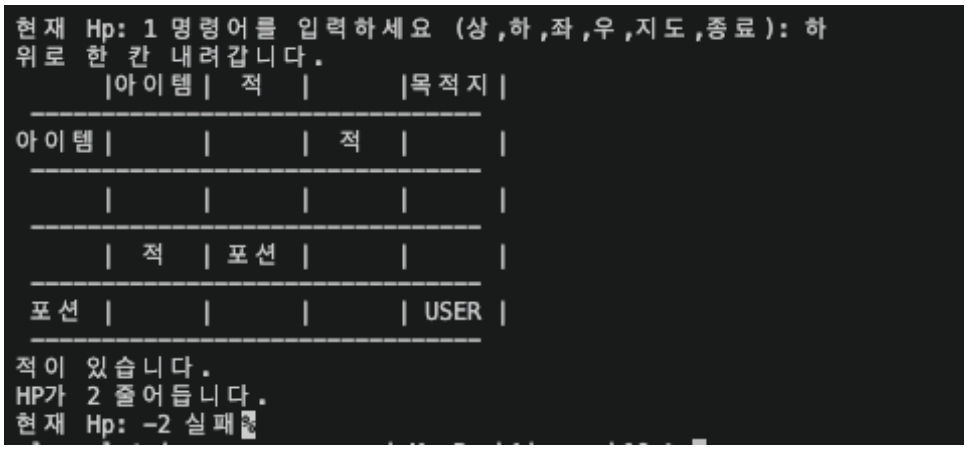


moveUP: 상 입력후 첫 시작 위치에서 위로 한칸 이동 확인



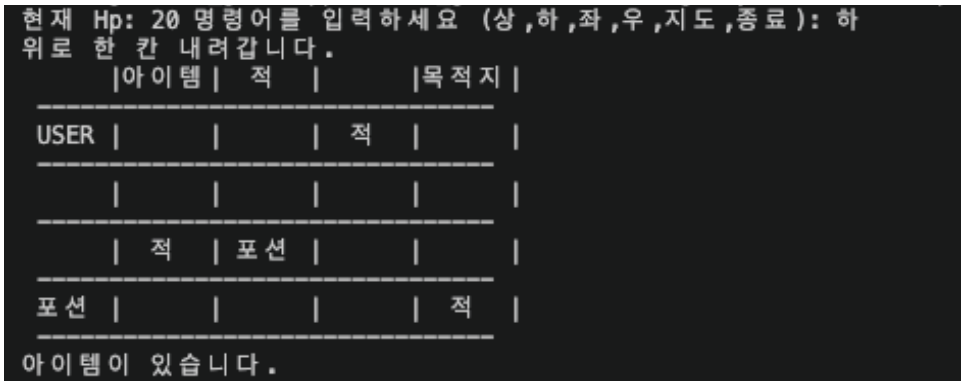
c. isValidUserHp

체력이 1미만이면 현재 체력 및 실패 출력 후 프로그램 종료

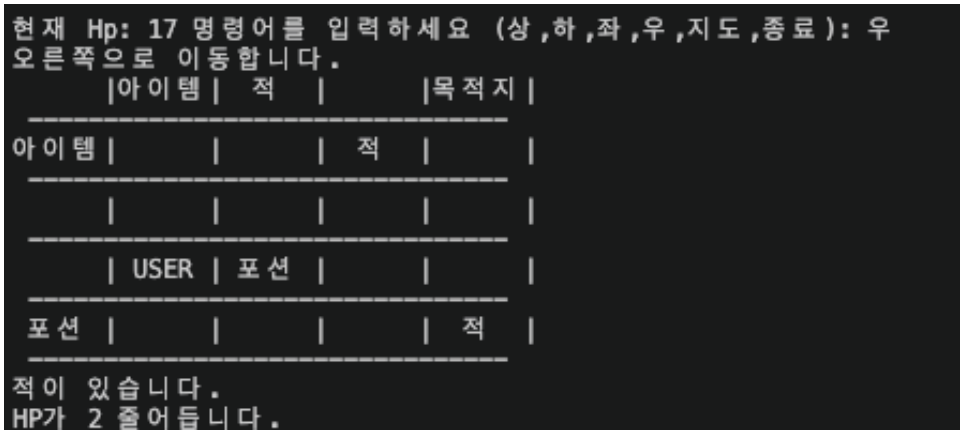


d. checkState

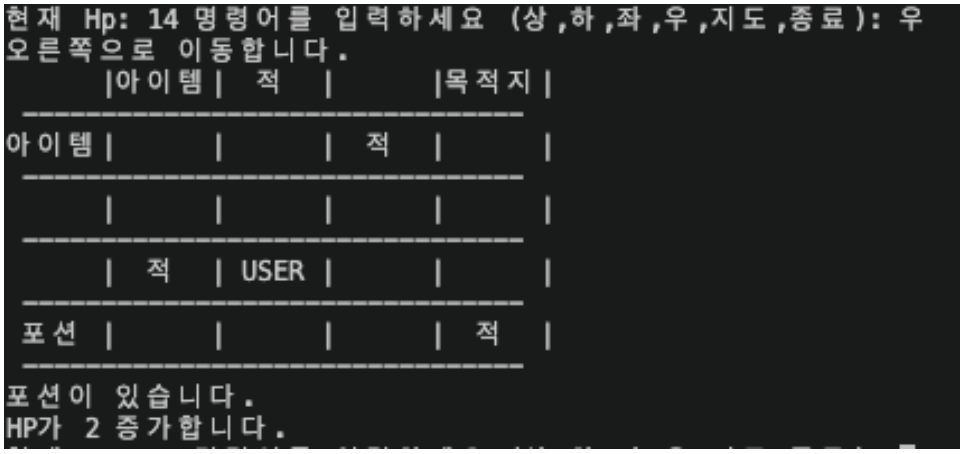
아이템 있는 칸



적이 있는 칸: 도착 후 HP 2 감소

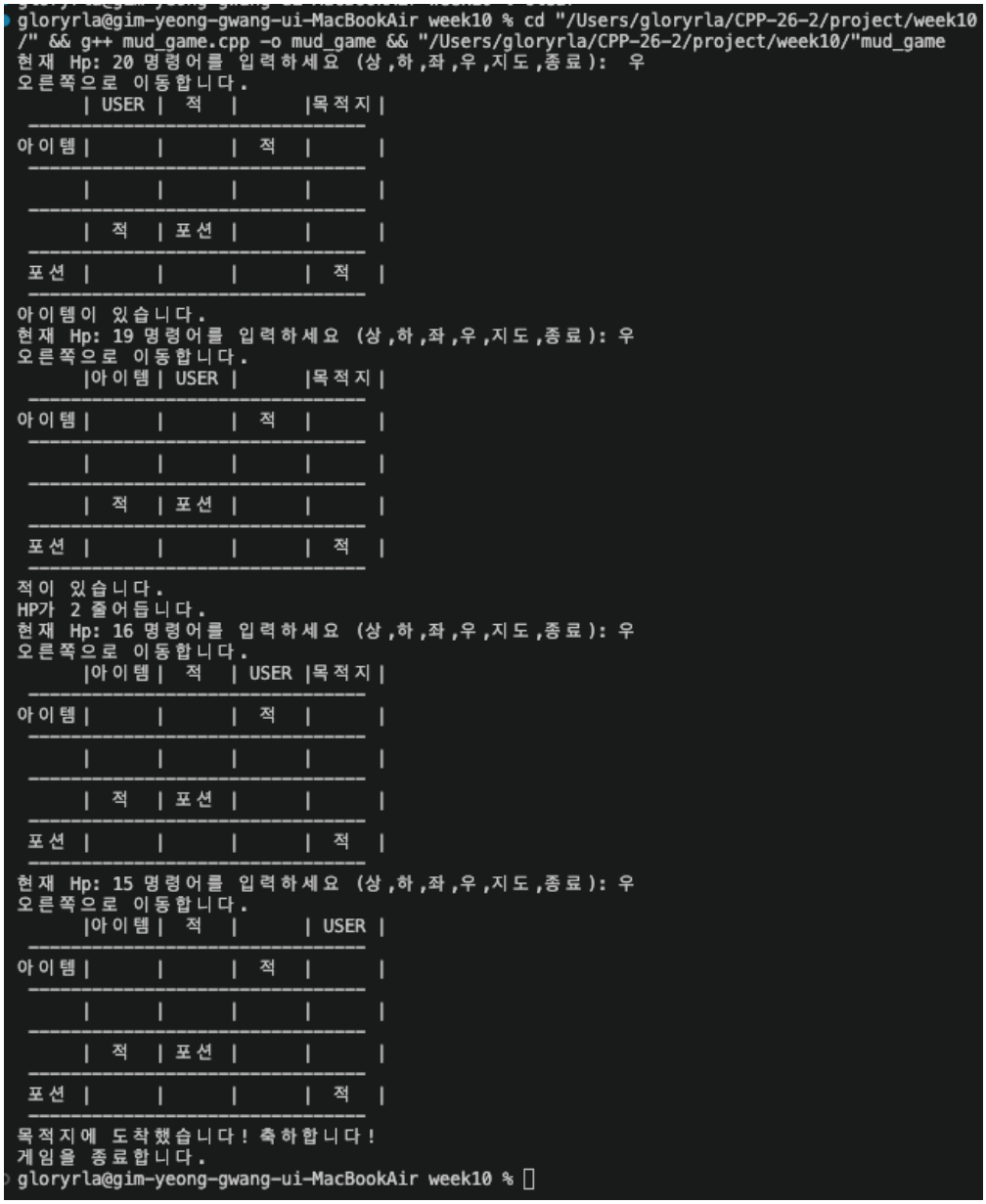


포션이 있는 칸: 도착 후 HP 2 증가



5. 결과 및 결론

5.1 프로젝트 결과



5.2 느낀점

새로운 기능을 만들땐 생각보다 기존에 코드들을 더 이해해야한다는 점을 느꼈습니다. 그리고 지금까지 배운 것을 토대로 이렇게 문제를 받고 고민하고 기능을 새로 만드는 것이 재밌었습니다. AI시대에 코드를 직접 칠일이 없는데 이번 기회로 코드 짜는 재미를 한번 더 느끼게 되어서 좋았습니다. 그리고 코드 함수화하는 과정에서 어디부분에서 어디까지 함수화로 묶어야하는지 고민이 많았습니다. 메인 함수에서 보기 쉽게 함수화를 해야할지 아니면 추상화를 가능하게 하는 것이 더 중점으로 해야하는지 고민을 했는데 아직 추상화하여 함수를 만드는 것은 어려운 것 같습니다. 그리서 이번에는 메인 함수 코드를 간략하게 만드는 것을 목표로 잡았습니다. 그리고 코드를 짜는건 재밌다 해도 보고서까지는 재밌어하진 않는구나 내가 보고서 작성을 지루해하구나를 느끼게 되었습니다.

앞으로 직장에 따라 보고서 작성 빈도가 달라질 수 있을텐데 이번 계기로 보고서 많이 작성하는 직장엔 별로 희망하지 않게 되는 저 자신을 찾을 수 있게 되었습니다.